

Threads e Concorrencia em Python

Sistema Bancario Concorrente

Codigo • Explicacao Tecnica • Arquitetura • Guia de Apresentacao • Perguntas e Respostas

3 THREADS

LOCK

QUEUE

MEM.
COMPARTILHADA

RACE CONDITION

Seção	Conteúdo
1	Cenário e Problema
2	Explicação Técnica Detalhada do Código
3	Arquitetura do Sistema
4	Por Que Este Código Se Encaixa no Trabalho
5	Código Completo Comentado

6	Como Rodar o Código ao Vivo
7	Guia de Apresentação — Estrutura do Pitch (5 min)
8	Perguntas e Respostas — Guia Completo para Todos os Integrantes

1. Cenário e Problema

1.1 O Problema do Mundo Real

Em sistemas bancários reais, múltiplos usuários acessam a mesma conta simultaneamente: um pelo aplicativo mobile, outro no caixa eletrônico, outro na agência. Quando todas essas operações chegam ao servidor ao mesmo tempo, elas disputam o mesmo recurso — o saldo da conta. Sem um mecanismo de controle, o sistema pode produzir resultados incorretos e causar prejuízo financeiro real.

1.2 O Que é Race Condition?

Race condition ocorre quando duas ou mais threads leem e escrevem uma variável compartilhada ao mesmo tempo sem sincronização. O resultado final depende de qual thread chegou primeiro — e não-determinístico: cada execução pode produzir um resultado diferente.

Exemplo concreto de Race Condition

Conta com R\$1.000. Thread A e Thread B querem sacar R\$100 cada.

Passo 1: Thread A lê saldo = 1000.

Passo 2: Thread B lê saldo = 1000 (A ainda não escreveu!).

Passo 3: Thread A calcula $1000 - 100 = 900$ e escreve 900.

Passo 4: Thread B calcula $1000 - 100 = 900$ e escreve 900.

Resultado: R\$900. Correto seria R\$800. O saque de A foi ignorado por B.

2. Explicação Técnica Detalhada do Código

2.1 ContaBancaria — Memória Compartilhada

O objeto ContaBancaria é criado uma única vez e passado para todas as threads. Em Python, objetos mutáveis passados como argumento não são copiados — a thread recebe o endereço de memória. Logo, conta.saldo é o mesmo valor para todas as threads.

Estrutura da classe

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.saldo = saldo_inicial # MEMÓRIA COMPARTILHADA – todas as threads acessam
        self.lock = threading.Lock() # LOCK – protege o saldo contra race condition
        self.historico = []
```

2.2 O Lock — Exclusão Mútua

O `threading.Lock()` funciona como uma trava: apenas uma thread por vez pode segurar o lock. Quando uma thread entra no bloco `with self.lock:`, ela adquire a trava. Outras threads que tentarem entrar ficam bloqueadas até o lock ser liberado.

Metodos protegidos por Lock

```
def depositar(self, valor, origem):
    with self.lock: # Thread adquire o lock – outras ficam bloqueadas
        self.saldo += valor # Operação atômica e segura
    # Lock liberado automaticamente ao sair do bloco

def sacar(self, valor, origem):
    with self.lock: # Mesma proteção
        if self.saldo >= valor:
            self.saldo -= valor
        return True
    return False
```

O que o 'with self.lock:' garante?

1. **Atomicidade:** leitura + cálculo + escrita do saldo sem interrupção.
2. **Visibilidade:** ao liberar o lock, a escrita fica visível para todas as threads.
3. **Segurança de exceções:** o lock é liberado automaticamente mesmo em caso de erro.

2.3 A Queue — Comunicação Entre Threads

A `queue.Queue()` é uma fila thread-safe. Ela implementa seus próprios locks internamente, permitindo uso simultâneo por múltiplas threads sem risco. É o canal de comunicação entre as threads produtoras (agências) e a thread consumidora (processador).

Padrão Produtor-Consumidor

```
# Thread PRODUTORA – enfileira transações
def agencia_online(conta, fila, nome):
```

```

for tipo, valor in operacoes:
    fila.put({'tipo': tipo, 'valor': valor, 'agencia': nome}) # thread-safe
fila.put(None) # sinal de encerramento

# Thread CONSUMIDORA – processa da fila
def processador_central(conta, fila, total_agencias):
    finalizadas = 0
    while finalizadas < total_agencias:
        tx = fila.get() # bloqueia se fila vazia – sem consumir CPU
        if tx is None: # sinal de encerramento recebido
            finalizadas += 1; continue
        conta.depositar(tx['valor'], tx['agencia']) # aplica com Lock

```

2.4 Coordenacao das Threads com join()

Criacao, inicio e sincronizacao das threads

```

fila = Queue()
conta = ContaBancaria('Joao Silva', 1000.00)

agencia1 = threading.Thread(target=agencia_online, args=(conta, fila, 'Agencia Rio'))
agencia2 = threading.Thread(target=agencia_online, args=(conta, fila, 'App Mobile'))
processador = threading.Thread(target=processador_central, args=(conta, fila, 2))

processador.start() # consumidora: ja aguarda na fila
agencia1.start() # produtora 1
agencia2.start() # produtora 2 – executa simultaneamente com agencia1

agencia1.join() # main espera agencia1 terminar
agencia2.join() # main espera agencia2 terminar
processador.join() # main espera processador terminar

```

O `join()` faz a thread principal aguardar cada thread terminar. Sem ele, o programa encerraria imediatamente apos `start()`, matando todas as threads no meio do trabalho.

3. Arquitetura do Sistema

3.1 Padrao Produtor-Consumidor

O sistema implementa o padrao classico de concorrencia Produtor-Consumidor. As threads produtoras geram dados (transacoes) e as colocam numa fila. A thread consumidora retira os dados e os processa. A Queue e o ponto de encontro entre os dois lados, garantindo que nenhuma transacao seja perdida.

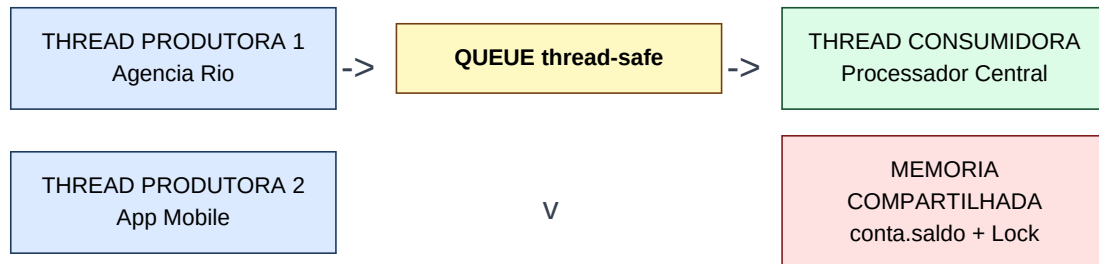


Figura 1 — Arquitetura do Sistema Bancario Concorrente

3.2 Fluxo de uma Transacao (passo a passo)

1	Agencia (produtora) cria dicionario: {'tipo': 'saque', 'valor': 200, 'agencia': 'App Mobile'}.
2	Chama fila.put(transacao) — transacao entra na Queue de forma thread-safe.
3	Processador esta bloqueado em fila.get(), aguardando item disponivel.
4	Item chega: processador acorda e chama conta.sacar() ou conta.depositar().
5	Dentro do metodo, a thread adquire o Lock antes de tocar no saldo.
6	Saldo atualizado atomicamente. Lock liberado. Operacao registrada no historico.
7	Processador chama fila.task_done() e volta a aguardar no get().

4. Por Que EsteCodigo Se Encaixa no Trabalho Proposto

4.1 Mapeamento Requisito por Requisito

Requisito	Atendido?	Onde no codigo
Minimo de 2 threads	SIM	agencia1, agencia2, processador
Compartilhamento de memoria	SIM	ContaBancaria.saldo (heap compartilhado)
Uso de Lock	SIM	threading.Lock() + with self.lock:
Uso de Queue	SIM	queue.Queue() – produtor-consumidor
Demonstracao pratica	SIM	main() executa ao vivo no terminal
Race condition demonstrada	SIM	demonstrar_race_condition()

4.2 Analise de Cada Requisito

Minimo de 2 Threads

O sistema possui 3 threads simultaneas: agencia1 (Thread Produtora 1 — Agencia Rio), agencia2 (Thread Produtora 2 — App Mobile) e processador (Thread Consumidora). As tres executam verdadeiramente em paralelo: agencia1 e agencia2 geram transacoes ao mesmo tempo enquanto processador ja consome. Isso e verificavel no terminal — as mensagens das agencias aparecem intercaladas, nao em sequencia.

Compartilhamento de Memoria

O objeto ContaBancaria (especialmente conta.saldo) e a memoria compartilhada. Criado uma unica vez com conta = ContaBancaria(...), e passado por referencia para as 3 threads. Todas operam sobre o mesmo endereco de memoria. A demonstracao de race condition prova isso: sem lock, o resultado e errado porque threads diferentes interferem no mesmo saldo.

Uso de Lock

O threading.Lock() e instanciado dentro de ContaBancaria e usado nos metodos depositar() e sacar() via 'with self.lock:'. Isso garante exclusao mutua: apenas uma thread por vez executa a secao critica. O 'with' e preferivel a acquire()/release() manual porque garante liberacao automatica mesmo em caso de excecao.

Uso de Queue

A queue.Queue() e o mecanismo de troca de dados. Produtoras chamam fila.put() e a consumidora chama fila.get(). A Queue garante: (1) nenhuma transacao perdida; (2) processador aguarda automaticamente quando vazia; (3) thread-safety interna sem lock adicional. O sinal None ao final indica encerramento — padrao elegante e seguro.

Demonstracao Pratica Funcionando

O codigo e executavel imediatamente com 'python banco_concorrente.py'. Produz: (1) demonstracao de race condition com resultado incorreto visivel; (2) sistema seguro com 12 operacoes em tempo real, mostrando cada transacao com saldo atualizado. O resultado da race condition muda a cada execucao — isso e concorrencia real nao-deterministica.

5. Código Completo Comentado

Imports

```
import threading # modulo de threads
import time # simular tempo de processamento
import random # variar tempo entre transacoes
from queue import Queue # fila thread-safe
```

Classe ContaBancaria — memoria compartilhada + lock

```
class ContaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.saldo = saldo_inicial # MEMORIA COMPARTILHADA
        self.lock = threading.Lock() # protecao
        self.historico = []

    def depositar(self, valor, origem):
        with self.lock: # 1 thread por vez
            self.saldo += valor

    def sacar(self, valor, origem):
        with self.lock: # 1 thread por vez
            if self.saldo >= valor:
                self.saldo -= valor; return True
            return False
```

Thread Produtora — agencia_online()

```
def agencia_online(conta, fila_transacoes, nome_agencia):
    operacoes = [('deposito',500),('saque',200),('deposito',1200),
                  ('saque',800),('deposito',300),('saque',1500)]
    for tipo, valor in operacoes:
        fila_transacoes.put({'tipo':tipo, 'valor':valor, 'agencia':nome_agencia})
        time.sleep(random.uniform(0.05, 0.15))
    fila_transacoes.put(None) # sinal de fim
```

Thread Consumidora — processador_central()

```
def processador_central(conta, fila_transacoes, total_agencias):
    finalizadas = 0
    while finalizadas < total_agencias:
        tx = fila_transacoes.get() # bloqueia se vazia
        if tx is None:
            finalizadas += 1; continue
        if tx['tipo'] == 'deposito': conta.depositar(tx['valor'], tx['agencia'])
        else: conta.sacar(tx['valor'], tx['agencia'])
        fila_transacoes.task_done()
```

Race condition — sem lock (demonstracao do problema)

```
def transferencia_sem_protecao(saldo_ref, valor, nome):
    lido = saldo_ref[0] # le saldo
    time.sleep(0.001) # JANELA: outra thread pode ler aqui!
    saldo_ref[0] = lido - valor # escreve baseado no valor antigo
```

main() — criacao e execucao

```
def main():
    conta = ContaBancaria('Joao Silva', saldo_inicial=1000.00)
    fila = Queue()
    agencia1 = threading.Thread(target=agencia_online,
                                args=(conta, fila, 'Agencia Rio'))
    agencia2 = threading.Thread(target=agencia_online,
                                args=(conta, fila, 'App Mobile'))
    processador = threading.Thread(target=processador_central,
                                    args=(conta, fila, 2))
    processador.start(); agencia1.start(); agencia2.start()
    agencia1.join(); agencia2.join(); processador.join()
```

6. Como Rodar o Código ao Vivo

Nenhuma instalação extra necessária

O código usa apenas bibliotecas que já vem com o Python: `threading`, `queue`, `time` e `random`. Qualquer Python 3.6 ou acima funciona. Para confirmar: abra o terminal e digite `'python --version'`.

Windows	Pressione Win+R, digite <code>cmd</code> e Enter. Navegue até a pasta com <code>cd caminho\pasta</code>
Mac	Command+Espace, digite <code>terminal</code> e Enter. Navegue com <code>cd caminho/pasta</code>
Linux	Ctrl+Alt+T. Navegue com <code>cd caminho/pasta</code>

Comando para executar:

```
python banco_concorrente.py
# ou, se necessário:
python3 banco_concorrente.py
```

O que aparece no terminal:

```
# --- PARTE 1: race condition (resultado errado) ---
[Thread-1] Retirou R$100.00 (sem lock) -> saldo: R$900.00
[Thread-2] Retirou R$100.00 (sem lock) -> saldo: R$900.00
Saldo final (esperado R$500.00): R$800.00 <- ERRADO!

# --- PARTE 2: sistema seguro (resultado correto) ---
[Agencia Rio] Enfileirou: DEPOSITO R$500.00
[App Mobile] Enfileirou: DEPOSITO R$500.00
[DEPOSITO] Agencia Rio | +R$500.00 | Saldo: R$1500.00
[DEPOSITO] App Mobile | +R$500.00 | Saldo: R$2000.00
...
```

7. Guia de Apresentacao — Estrutura do Pitch (5 minutos)

Fala de abertura — diga algo nessa linha, sem decorar

"Nosso sistema simula um banco onde multiplos canais — agencia e app mobile — fazem operacoes ao mesmo tempo na mesma conta. O problema e que sem controle, duas operacoes simultaneas podem corromper o saldo. A gente vai mostrar esse bug acontecendo ao vivo, e depois a solucao com threads, lock e queue."

1 min

Cenario / Problema

Execute o codigo. Mostra a race condition ao vivo. Fala: 'Conta com R\$1.000, 5 saques de R\$100 — esperado R\$500. Vejam o que acontece sem controle.' O resultado incorreto no terminal ja e a prova do problema.

2 min

Explicacao Tecnica

Abra o codigo. Aponte para `self.saldo`: 'Isso e a memoria compartilhada — todas as threads enxergam o mesmo objeto.' Aponte o `with self.lock`: e explique exclusao mutua. Mostre a Queue e o padrao produtor-consumidor.

1 min

Demonstracao

Execute `main()` ao vivo. Mostra as mensagens intercaladas no terminal — isso e concorrencia real. Aponta o saldo sendo atualizado corretamente em cada operacao.

1 min

Perguntas

Prováveis: GIL do Python (nao elimina race conditions em operacoes compostas), diferenca Lock vs Queue (Lock para exclusao mutua; Queue para comunicacao), por que `join()` (para esperar as threads terminarem).

8. Perguntas e Respostas — Guia para Todos os Integrantes

Por que esta secao existe?

O professor pode sortear qualquer integrante para responder perguntas sobre threads, memoria compartilhada, sincronizacao e conceitos abordados em aula. Quem nao souber responder gera penalizacao para toda a equipe. Leia esta secao — uma leitura ja e suficiente para responder tudo com seguranca.

Sobre Threads

O que e uma thread?

E uma linha de execucao dentro de um programa. Quando voce tem duas threads, o computador executa as duas ao mesmo tempo — ou alternando muito rapidamente, dando impressao de simultaneidade. No nosso codigo, a Agencia Rio e o App Mobile sao threads que rodam juntas.

Qual a diferenca entre thread e processo?

Processo e um programa completo com sua propria memoria isolada. Thread e uma subdivisao dentro de um processo que COMPARTILHA a memoria com as outras threads do mesmo processo. E por isso que threads conseguem compartilhar o saldo da conta — elas estao no mesmo processo.

Por que usamos 3 threads e nao so uma?

Porque no mundo real as operacoes chegam ao mesmo tempo. Com uma thread so, ela processaria uma operacao de cada vez — seria sequencial, nao concorrente. Com 3 threads, duas agencias geram transacoes simultaneamente enquanto o processador ja esta trabalhando.

Como voce sabe que as threads realmente rodam ao mesmo tempo?

As mensagens no terminal aparecem intercaladas — 'Agencia Rio enfileirou...' e 'App Mobile enfileirou...' aparecem misturadas, nao em blocos separados. Isso acontece porque as duas threads estao ativas ao mesmo tempo e o sistema operacional alterna entre elas.

Sobre Memoria Compartilhada

Onde esta a memoria compartilhada no codigo?

E o objeto ContaBancaria, especificamente o self.saldo. Ele e criado uma vez e passado como argumento para as 3 threads. Em Python, quando voce passa um objeto mutavel para uma funcao, nao e feita uma copia — todas as threads recebem o endereco do mesmo objeto na memoria. Quando qualquer thread muda o saldo, todas as outras enxergam o novo valor.

O que acontece se nao houver memoria compartilhada?

Cada thread teria sua propria copia do saldo. As operacoes de uma nao afetariam a outra. O banco teria saldos diferentes dependendo de qual versao voce consultasse — seria impossivel ter um saldo unico e consistente para a mesma conta.

Por que saldo_ref = [1000.0] usa lista e nao so saldo = 1000.0?

Em Python, numeros sao imutaveis. Se voce passa um float para uma funcao e faz saldo = saldo - 100, ela cria um novo objeto local — nao modifica o original. Usando uma lista [1000.0], a funcao acessa saldo_ref[0] que e uma referencia mutavel ao mesmo objeto. Isso simula passagem por referencia para demonstrar a race condition.

Sobre Lock e Sincronizacao

O que e um Lock e para que serve?

Lock e uma trava. Quando uma thread entra no bloco 'with self.lock:', ela adquire essa trava. Qualquer outra thread que tente entrar no mesmo bloco fica bloqueada — pausada — ate a primeira sair e liberar a trava. Isso garante que so uma thread por vez modifica o saldo.

O que e exclusao mutua?

E o que o lock garante: que duas threads nao executem a mesma secao critica ao mesmo tempo. 'Mutua' porque nenhuma das duas entra — elas se excluem mutuamente daquele trecho de codigo. No nosso caso, a secao critica e a leitura e escrita do self.saldo.

O que e secao critica?

E o trecho de codigo que acessa um recurso compartilhado e que, se executado por duas threads ao mesmo tempo, causa problema. No nosso codigo, a secao critica e a sequencia: ler saldo -> calcular novo valor -> escrever novo saldo.

Por que usamos 'with self.lock:' e nao lock.acquire() e lock.release() manual?

O 'with' garante que o lock seja liberado automaticamente mesmo se ocorrer uma excecao no meio do bloco. Se usassemos acquire/release manual e desse um erro antes do release, o lock nunca seria liberado — o programa travaria para sempre. O 'with' e mais seguro e mais legivel.

O que acontece se duas threads tentam adquirir o mesmo lock ao mesmo tempo?

Apenas uma consegue. A primeira que chegar adquire o lock e continua executando. A segunda fica bloqueada (pausada pelo sistema operacional) ate o lock ser liberado. Quando a primeira sair do bloco 'with', o lock e liberado e a segunda pode entrar.

Sobre Queue

O que e uma Queue e por que ela e thread-safe?

Queue e uma fila. O Python ja implementa internamente os locks necessarios dentro da Queue, entao voce nao precisa sincronizar o acesso a ela. Varias threads podem chamar put() e get() ao mesmo tempo sem risco de corrupcao de dados.

O que e o padrao Produtor-Consumidor?

E um padrao de design para concorrencia. O produtor gera dados e coloca numa fila. O consumidor pega os dados da fila e processa. Eles nao precisam saber da existencia um do outro — a fila e o intermediario. No nosso codigo, as agencias sao produtoras e o processador central e o consumidor.

Por que o processador usa fila.get() dentro de um loop?

Porque o get() bloqueia automaticamente quando a fila esta vazia — o processador fica pausado esperando, sem consumir CPU. Quando uma agencia coloca algo na fila, o processador acorda sozinho. O loop continua ate todas as agencias enviarem o sinal None de encerramento.

Para que serve o None no final de cada agencia?

E o sinal de encerramento. Quando o processador recebe um None, ele sabe que aquela agencia terminou. Quando recebe None de todas as agencias (controlado pela variavel 'finalizadas'), ele sai do loop e encerra. E uma convencao simples e eficiente para sinalizar fim de dados.

Qual a diferenca entre usar Lock direto e usar Queue?

Lock e para exclusao mutua em um recurso compartilhado — garante que so uma thread modifica o saldo por vez. Queue e para comunicacao entre threads — permite que uma thread envie dados para outra de forma segura. No nosso sistema usamos os dois: Queue para enviar transacoes entre agencias e processador, Lock para proteger o saldo no momento da escrita.

Sobre Race Condition

O que é race condition?

É quando o resultado de uma operação depende da ordem em que as threads executam, e essa ordem é imprevisível. No nosso exemplo sem lock: Thread A lê o saldo 1000, Thread B também lê 1000 antes de A escrever, A escreve 900, B escreve 900 — o segundo saque foi ignorado. O resultado muda a cada execução.

Como o código demonstra a race condition?

A função `demonstrar_race_condition()` usa `time.sleep(0.001)` para criar uma janela de tempo onde outra thread consegue ler o mesmo saldo antes da primeira escrever. O resultado incorreto aparece no terminal — normalmente R\$800 ou R\$900 em vez de R\$500.

Por que o resultado da race condition muda a cada execução?

Porque o sistema operacional é quem decide quando cada thread executa. Essa decisão depende de carga do processador, prioridades, eventos do sistema — fatores que mudam a cada segundo. É por isso que race condition é tão perigosa: ela não acontece sempre, dificultando a detecção.

Sobre `join()` e Coordenação

Para que serve o `thread.join()`?

Faz a thread que chamou (a main) esperar a thread-alvo terminar antes de continuar. Sem `join()`, o programa principal chegaria ao final e encerraria, matando todas as threads no meio do trabalho. Com `join()`, garantimos que o resultado final só é exibido depois que todas as operações terminaram.

Por que o processador é iniciado antes das agências?

Para garantir que a thread consumidora já esteja aguardando na fila quando as agências comecem a enfileirar transações. Se as agências começarem antes, elas já colocariam itens na fila antes do processador estar pronto — o que funciona porque a Queue guarda os itens, mas é uma boa prática iniciar o consumidor primeiro.

Perguntas Armadilha — Leia Com Atenção

Estas perguntas parecem simples mas escondem uma pegadinha. São marcadas em vermelho pois o professor frequentemente as usa para testar o entendimento real.

ARMADILHA — O GIL do Python não já resolve o problema de concorrência?

NAO. O GIL (Global Interpreter Lock) garante que só uma thread executa bytecode Python por vez, mas ele NAO garante atomicidade em operações compostas. A sequência 'ler saldo -> calcular -> escrever saldo' são três operações separadas. O GIL pode ser liberado entre qualquer uma delas, permitindo que outra thread entre no meio. Por isso o Lock explícito ainda é necessário — a race condition demonstrada no código prova que o GIL não é suficiente.

ARMADILHA — Threads em Python rodam verdadeiramente em paralelo?

Depende. O GIL impede que threads Python executem código Python puro em múltiplos núcleos simultaneamente. Porém, operações de I/O (leitura de arquivo, requisição de rede, espera) liberam o GIL, permitindo paralelismo real. Para o nosso sistema bancário, o `time.sleep()` simula I/O e libera o GIL — portanto as threads SIM executam em paralelo nesse contexto, e é por isso que a race condition acontece.

ARMADILHA — O que é deadlock e nosso código pode ter um?

Deadlock é quando duas threads ficam esperando uma pela outra para sempre — nenhuma consegue continuar. Exemplo: Thread A tem Lock1 e espera Lock2; Thread B tem Lock2 e espera Lock1. Nosso código NAO tem deadlock porque cada método usa apenas um lock (`self.lock`) e libera imediatamente ao sair do bloco 'with'. Não há situação de dois locks sendo adquiridos em ordens diferentes.

Resumo Rápido —

Conceito	Frase curta
Thread	Linha de execução paralela dentro do programa
Memoria compartilhada	Mesmo objeto na memória acessado por várias threads
Lock	Trava que deixa só uma thread passar por vez
Seção crítica	Trecho que não pode ser executado em paralelo
Queue	Fila thread-safe para comunicação entre threads
Race condition	Resultado incorreto por falta de sincronização
<code>join()</code>	Thread principal espera a outra terminar
Produtor-Consumidor	Uma thread gera dados, outra processa via fila
Deadlock	Dois threads esperando uma pela outra — programa trava
GIL	Lock do Python — não substitui Lock explícito em operações compostas

Link do código do GITHUB :

<https://github.com/franklin808/Thiago-UVA/tree/main>

Alunos : Franklin Gaio Figueira Filho -
1230115176 Maria Luiza França Mendes -
1230112799 Jorge Luiz Furtado Maia -
1230113709 Daniela Moreira Negreiros Dias -
1230203973